DT

DualThread

Two threads. One conversation.

WINDOWS C++ DESKTOP CHAT

Step-by-Step Execution Guide

A complete, repeatable procedure for extracting, starting, verifying and demonstrating the DualThread desktop chat stack: MFC clients, dual chat servers, load balancer, Redis, Kafka and MySQL.

Release x64 | MFC client | Kafka + Redis + MySQL | Native ODBC

Delivery package: DualThread-delivery.zip

Version 2.0 · 22 October 2026

Quick start checklist

For a working machine with MySQL, Memurai and Kafka already installed. Full details follow on the next pages.

- [ ] Use `DualThread-delivery.zip` — **not** the old 71 KB `CppChatMessaging.zip`.
- [ ] Extract it to `C:\DualThread`.
- [ ] Close old clients and stop old DualThread services.
- [ ] Make sure MySQL, Memurai and Kafka are running.
- [ ] Run `scripts\start-all.ps1`.
- [ ] Confirm ports **5000**, **6001**, **6002** and **9092**.
- [ ] Open two `ChatClient.exe` windows.
- [ ] Sign in as **user1/pass123** and **user2/pass123**.
- [ ] Send a personal message, then create a group.

Five commands that do most of the work

```
Set-Location "C:\DualThread\DualThread-delivery"
powershell -ExecutionPolicy Bypass -File .\scripts\start-all.ps1
Start-Process "$PWD\bin\Release\ChatClient.exe" -WorkingDirectory $PWD
Start-Process "$PWD\bin\Release\ChatClient.exe" -WorkingDirectory $PWD
powershell -ExecutionPolicy Bypass -File .\scripts\stop-all.ps1
```

IMPORTANT

Stop old services before starting the extracted copy.

If ports 5000, 6001 or 6002 are already in use, `start-all.ps1` exits with a clear message. Run the stop script from the old installation first, then start the new extraction.

EXPECTED RESULT

A working two-user demo in under 10 minutes.

Both clients show *Connected*, presence dots, and messages flowing between two different backend servers.

What you need

Package

Use: `DualThread-delivery.zip` (about 2.1 MB)

Contains source, configs, database scripts, docs, screenshots, startup scripts, and prebuilt Release x64 binaries with DLLs.

Machine

Windows 10/11 x64, PowerShell 5.1 or later.

No build is required if you use the included `bin\Release` binaries.

Services and ports

SERVICE	PORT	PURPOSE
MySQL 8.0	**3306**	Users, groups and chat history
Memurai / Redis	**6379**	Auth cache, presence and Pub/Sub
Kafka broker	**9092**	Message pipeline to DBWriter
LoadBalancer	**5000** TCP / **5001** UDP	Client entry point and health checks
ChatServer 1	**6001** TCP / **7001** UDP	First chat backend
ChatServer 2	**6002** TCP / **7002** UDP	Second chat backend

Demo accounts

USER ID	PASSWORD	SUGGESTED USE
`user1`	`pass123`	First client
`user2`	`pass123`	Second client
`user3`	`pass456`	Optional third client / unread-badge test

POWERSHELL PERMISSIONS

Start-Service requires an elevated PowerShell window.

If MySQL, Memurai and Kafka are already running, you do *not* need administrator rights to start DualThread itself. Open PowerShell as Administrator only when you need to start a stopped Windows service.

DO NOT CONFUSE THE ARCHIVES

Use the 2,054 KB archive.

The 71 KB `CppChatMessaging.zip` is an older, incomplete source package. The final delivery is `DualThread-delivery.zip`.

Prepare the workspace

1 Open PowerShell as Administrator

Press **Win**, type **PowerShell**, right-click **Windows PowerShell** and choose **Run as administrator**.

You only need elevation if MySQL or Memurai is stopped. Elevation also lets `start-all.ps1` start those services automatically.

2 Stop any old DualThread services

Run the stop script from the old installation first if it exists:

```
Set-Location "C:\assessment\Cpp-Chat-Messaging-System-Advanced-main"
powershell -ExecutionPolicy Bypass -File .\scripts\stop-all.ps1
```

Expected output:

```
LoadBalancer   stopped
ChatServer     stopped
ChatServer     stopped
DBWriter       stopped
DualThread application ports are free.
```

This stops DualThread processes only. MySQL, Memurai and Kafka are left running intentionally.

3 Extract the delivery ZIP

Extract to a clean folder. The commands below use `C:\DualThread`.

```
$zip  = "C:\assessment\DualThread-delivery.zip"
$dest = "C:\DualThread"

if (-not (Test-Path $dest)) {
    New-Item -ItemType Directory -Path $dest | Out-Null
}

Expand-Archive -Path $zip -DestinationPath $dest -Force

$root = Join-Path $dest "DualThread-delivery"
Set-Location $root
Get-ChildItem
```

You should see these folders:

```
bin  ChatClient  ChatServer  Common  config
database  DBWriter  docs  LoadBalancer  scripts
```

If you already extracted the ZIP, skip the extraction and set `$root` to the existing folder.

KEEP THIS POWERSHELL WINDOW OPEN

The variable `$root` is used in later commands.

If you open a new PowerShell window, set `$root` again before launching clients or restarting services.

Start the dependencies and services

4 Check MySQL and Memurai

```
Get-Service MySQL80,Memurai -ErrorAction SilentlyContinue |
    Format-Table Status,Name
```

Both should show **Running**. If either is stopped, start it from the elevated window:

```
Start-Service MySQL80
Start-Service Memurai
```

> **IF YOU SEE "CANNOT OPEN SERVICE"**
>
> **Your PowerShell is not elevated.**
>
> Close it, reopen PowerShell with **Run as administrator**, and try again. If the services are already **Running**, skip `Start-Service` entirely and continue.

Kafka does not need to be started manually. `start-all.ps1` checks port 9092 and starts Kafka from `C:\kafka` when needed.

5 Start DBWriter, both servers and the load balancer

```
Set-Location $root
powershell -ExecutionPolicy Bypass -File .\scripts\start-all.ps1
```

Expected output:

```
Kafka is already listening on 9092.
DBWriter       started (PID ...)
ChatServer1    started (PID ...)
ChatServer2    started (PID ...)
LoadBalancer   started (PID ...)

DualThread services are running:
  ChatServer 1  TCP 6001 / UDP 7001
  ChatServer 2  TCP 6002 / UDP 7002
  LoadBalancer  TCP 5000 / UDP 5001
```

If Kafka is installed elsewhere, use `-KafkaHome "D:\kafka"`. The launcher waits up to 60 seconds for port 9092.

> **WHAT THE SCRIPT DOES**
>
> **It creates `logs\`, starts the services hidden, and verifies the ports.**
>
> Service stdout/stderr is written to `logs\*.out.log` and `logs\*.err.log`. The application logs remain in `logs\server1.log`, `logs\server2.log`, `logs\dbwriter.log` and `logs\loadbalancer.log`.

Verify the stack and launch clients

6 Verify TCP ports and processes

```
Get-NetTCPConnection -State Listen -LocalPort 5000,6001,6002,9092 |
    Sort-Object LocalPort |
    Format-Table LocalAddress,LocalPort,OwningProcess
```

You should see four listening ports:

PORT	PROCESS
5000	LoadBalancer.exe
6001	ChatServer.exe — server1
6002	ChatServer.exe — server2
9092	Kafka broker

```
Get-Process DBWriter,ChatServer,LoadBalancer |
    Format-Table Id,ProcessName,StartTime
```

7 Open two chat clients

```
Start-Process "$root\bin\Release\ChatClient.exe" -WorkingDirectory $root
Start-Process "$root\bin\Release\ChatClient.exe" -WorkingDirectory $root
```

Two **DualThread — Sign in** windows open. Keep both open and move them side by side.

Optional third client for the unread-badge test:

```
Start-Process "$root\bin\Release\ChatClient.exe" -WorkingDirectory $root
```

WHAT SUCCESS LOOKS LIKE

The login window shows the DualThread branding and defaults to `127.0.0.1:5000`.

The client reads `config\client.ini` from the working directory, so always start it with `-WorkingDirectory $root`.

Ports and routing

The load balancer assigns each new client connection to a healthy backend in round-robin order. Two users may therefore land on different chat servers — this is expected. Cross-server messages are routed through Redis Pub/Sub.

Sign in

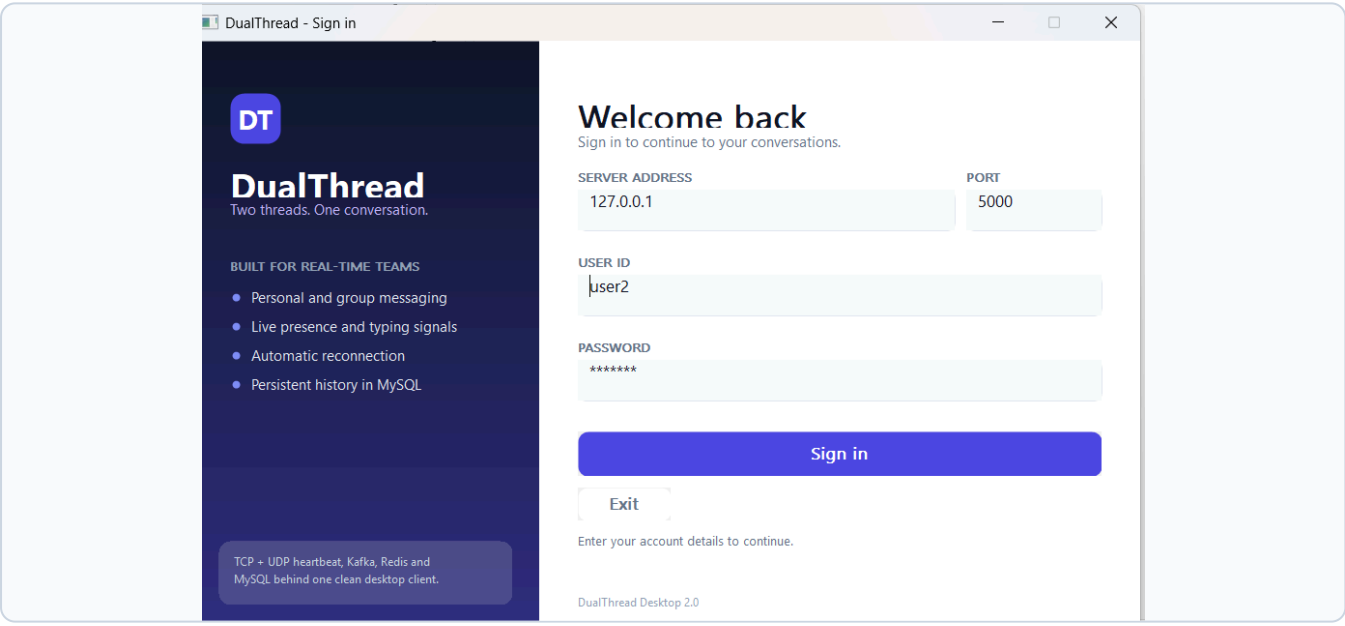
8 Use the demo credentials

In the first window:

FIELD	VALUE
Server	`127.0.0.1`
Port	`5000`
User ID	`user1`
Password	`pass123`

In the second window, use `user2` / `pass123`. Click **Sign in** in each window.

Expected: the main window opens with the title **DualThread — user1** or **DualThread — user2**, and the header shows **Connected**.



DualThread sign-in screen. Server and port default to 127.0.0.1:5000.

IF LOGIN FAILS

Check the service logs before changing anything.

Open `logs\server1.log` and `logs\server2.log`. A refused connection usually means the load balancer is not listening on 5000. An authentication error with valid credentials usually means the database was not initialized or the ODBC driver name does not match the installed driver.

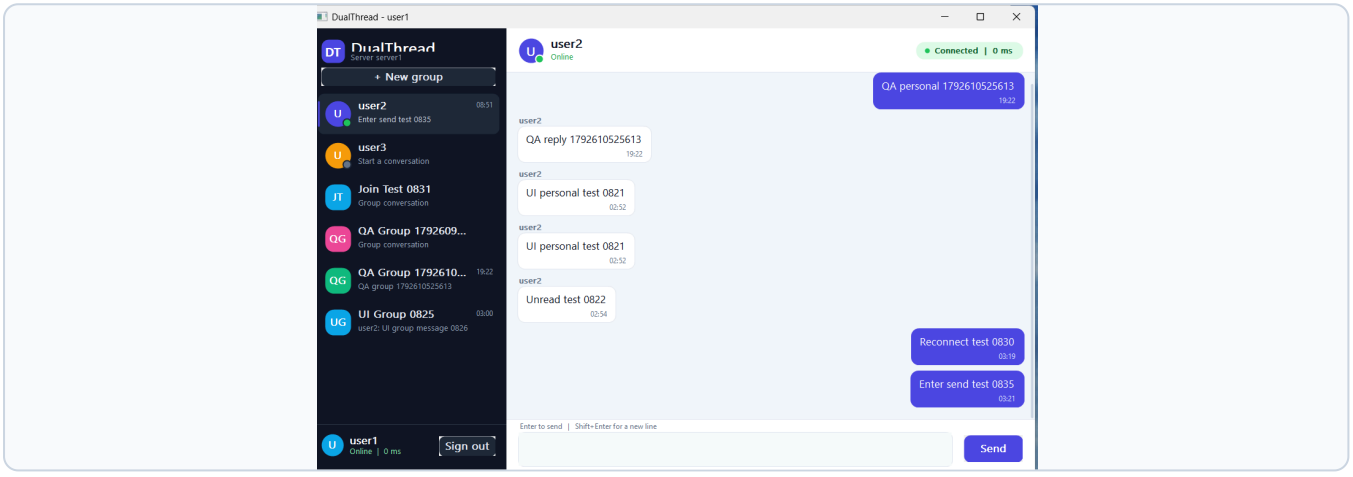
Personal messaging and unread badges

9 Send a personal message

In the `user1` window, select `user2` in the left conversation list. Type a message and press **Enter**:

```
Hello from user1
```

The message appears immediately in `user1`; `user2` receives it through the load balancer and the cross-server routing path. Reply from `user2` to verify both directions.



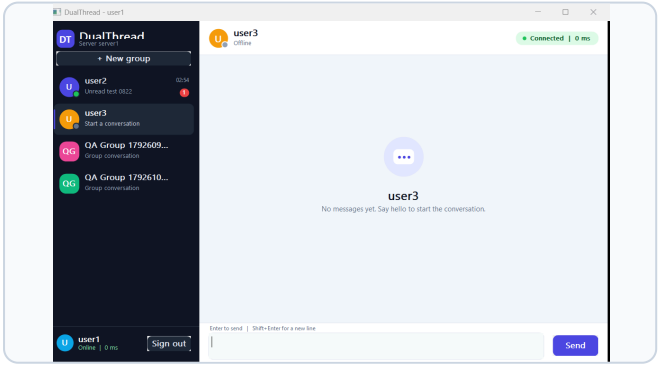
Personal conversation with aligned bubbles, timestamps, presence and the composer.

10 Test the unread badge

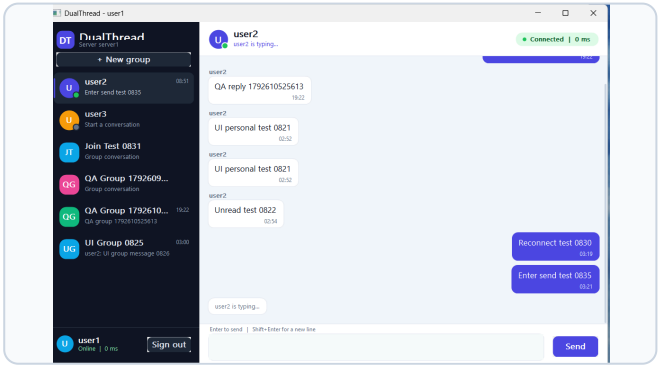
In `user1`, select `user3` (or any other conversation). From `user2`, send:

```
Unread test
```

A numeric badge appears on the `user2` conversation in `user1`. Select `user2`; the badge clears and the message is visible.



Unread badge on the conversation list.



Typing indicator while the other user composes.

EXPECTED BEHAVIOR

Enter sends; Shift+Enter inserts a newline.

The composer is multiline. While disconnected, the send button is disabled and the text is preserved.

Create and use a group

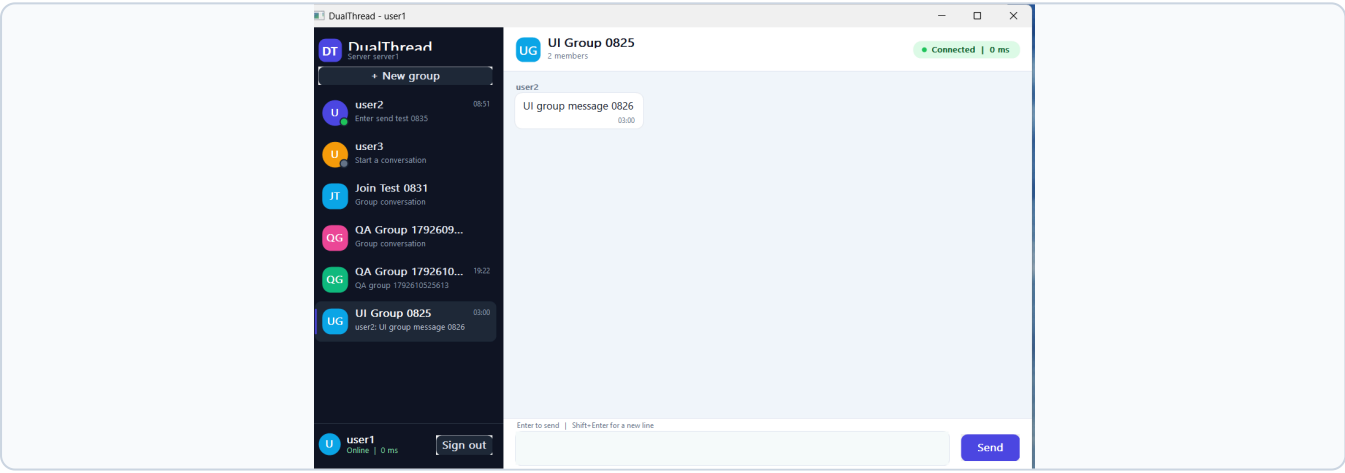
11 Create a group from the client

1. Click **+ New group**.
2. Choose **Create**.
3. Group name: `Demo Group`.
4. Members: `user2`.
5. Click the create/OK button.

The group appears in both clients. Open it in `user1`, send a message, and confirm that `user2` receives it.

```
Welcome to Demo Group
```

The creator is always a member. Membership is stored in `tbl_GroupMembers` and synchronized across servers through Redis.



Group conversation with the friendly group name, member count and message history.

JOIN AN EXISTING GROUP

Use the same + New group dialog and choose Join.

Paste the group ID and confirm. The group should appear with the member count and a “Joined group.” confirmation. A screenshot of the verified join flow is included in `docs\screenshots\group-joined.png`.

GROUP HISTORY

Group history is loaded from MySQL. Only members can request it. If history is empty immediately after creating a group, send one message and reopen the conversation.

Test automatic reconnection

12 Stop and restart the load balancer

With both clients signed in, stop only the load balancer:

```
Stop-Process -Name LoadBalancer -Force
```

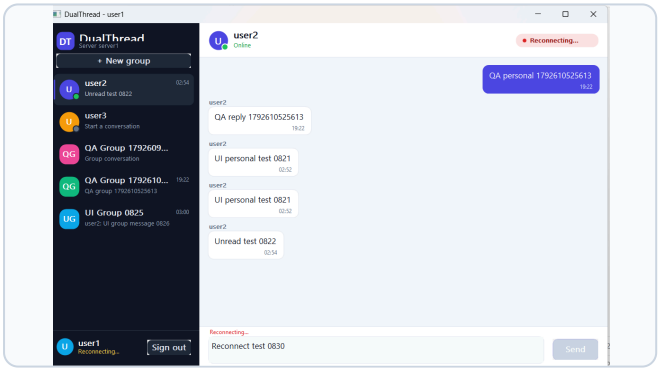
Within a few seconds, both clients should show:

- a red **Reconnecting...** status pill;
- a matching profile/status line;
- the send button disabled;
- the composer text preserved.

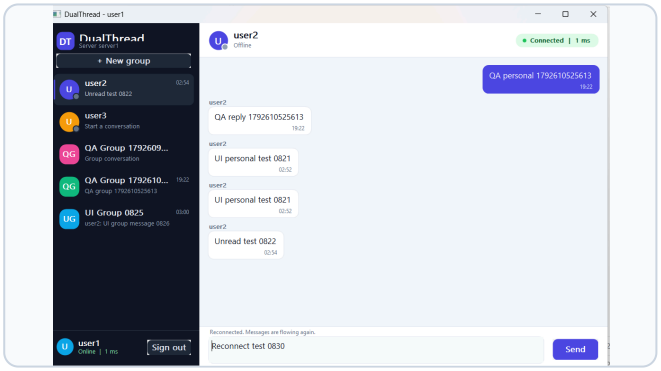
Restart it from the same PowerShell window:

```
Start-Process "$root\bin\Release\LoadBalancer.exe" `
    -ArgumentList "config\loadbalancer.ini" `
    -WorkingDirectory $root -WindowStyle Hidden
```

After recovery the header returns to **Connected**, the send button is enabled, and the client shows **Reconnected. Messages are flowing again.** Presence and selected history refresh automatically.



Connection lost: feedback and disabled send.



Recovered: connected and messages flowing.

WHAT THIS PROVES

The client survives a backend-entry failure without restarting.

It retries with a 1–10 second backoff, re-authenticates, and refreshes groups, presence and history.

Stop everything

Close both `ChatClient` windows first, then run:

```
Set-Location $root
powershell -ExecutionPolicy Bypass -File .\scripts\stop-all.ps1
```

Expected:

```
LoadBalancer   stopped
ChatServer     stopped
ChatServer     stopped
DBWriter       stopped
DualThread application ports are free.
```

This leaves MySQL, Memurai and Kafka running. To stop Kafka too:

```
powershell -ExecutionPolicy Bypass -File .\scripts\stop-all.ps1 -StopKafka
```

Troubleshooting

SYMPTOM	CHECK / FIX
"Cannot open MySQL80/Memurai service"	PowerShell is not elevated. Reopen as Administrator, or skip if the service is already Running.
Ports 5000/6001/6002 already in use	An old DualThread copy is still running. Run its `stop-all.ps1` first.
"Kafka did not start"	Check `logs\kafka.err.log`. Confirm Kafka is installed at `C:\kafka` or pass `-KafkaHome`.
Login says connection refused	LoadBalancer is not listening on 5000. Start the service stack again.
Login fails with valid credentials	Database is missing or demo users were not inserted. See the first-run commands below.
Messages arrive but are not stored	Kafka is down or DBWriter cannot reach MySQL. Check `logs\dbwriter.log`.
Messages show a raw group ID	Use the current build; the group-name regression was fixed. Restart the client from `bin\Release`.

FIRST-TIME DATABASE ONLY

Do not run `sample_data.sql` repeatedly on an existing database.

For a brand-new installation:

```
$mysql = "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe"
Set-Location $root
& $mysql -uroot -p -e "source database/schema.sql"
& $mysql -uroot -p -e "source database/sample_data.sql"
```

Optional: build from source

The delivery ZIP already contains Release x64 binaries. Build only if you change the source or need a different toolset.

A Prerequisites

- Visual Studio 2026 with Desktop C++ and MFC for v145.
- Windows 10/11 SDK.
- `VCPKG_ROOT` pointing to a vcpkg installation.
- vcpkg packages: `hiredis:x64-windows` and `librdkafka:x64-windows`.
- 64-bit MySQL Connector/ODBC. The verified driver name is `MySQL ODBC 26.7 ANSI Driver`.

The projects use toolset v145. Visual Studio 2022 users must retarget all four projects to v143.

B Install libraries and build

```
setx VCPKG_ROOT "C:\vcpkg"
# Open a new terminal after setx, or set it for this session:
$env:VCPKG_ROOT = "C:\vcpkg"

Set-Location C:\vcpkg
.\vcpkg install hiredis:x64-windows librdkafka:x64-windows

Set-Location $root
& "C:\Program Files\Microsoft Visual Studio\18\Community\MSBuild\Current\Bin\MSBuild.exe" `
  .\CppChatMessaging.sln `
  /t:Rebuild /p:Configuration=Release /p:Platform=x64 /m
```

Expected: `MSBUILD_EXIT=0`, with all four projects built into `bin\Release`.

DOCUMENTATION MAP

More detail is included in the delivery package.

`README.md`	Build, database, startup, protocol, troubleshooting
`docs\ARCHITECTURE.md`	Topology, flows, threading, storage, failure behavior
`docs\TRACEABILITY.md`	Requirement-to-evidence mapping
`docs\TEST_RESULTS.md`	Actual tests, SQL evidence and limitations
`docs\screenshots\`	Login, personal, group, typing, unread, reconnect

VERIFIED BASELINE

Release x64 rebuild exited 0.

Cross-server personal and group messaging, presence, typing, unread badges, history, group create/join, logout/re-login, reconnect recovery and MySQL persistence were exercised. Shift+Enter and resize/DPI remain manual checks.